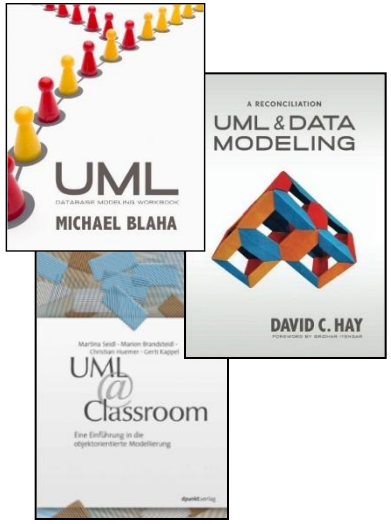
Modul 1

Datenbankmodellierung mit der Unified Modeling Language

Josef Altmann



Der vorliegende Foliensatz basiert auf:

M. Seidl, M. Brandsteidl, C. Huemer, G. Kappel: UML@Classroom, dpunkt.verlag, 2012.

M. Blaha: UML Database Modeling Workbook, Technics Publications, LLC, 2013.

D. C. Hay: UML&Data Modeling, Technics Publications, LLC, 2011.

Überblick

- Konzeptuelle Datenmodellierung
  - Modellbegriff
  - Datenmodelle
  - Datenbankentwurfsschritte
- Objektorientierte Datenmodelle
  - in UML
  - Konzepte der Objektorientierung
- Klassendiagramm
  - Klassen- vs. Objektdiagramm
  - Klassen
  - Beziehungen

- Anhang: Definitionen

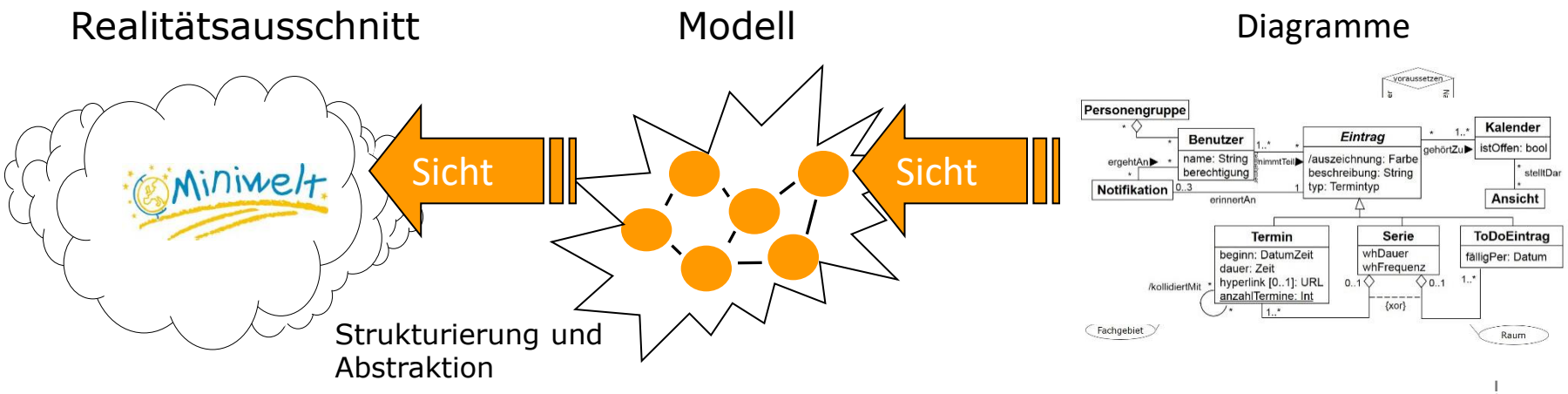
© 2025

Modellbegriff
Abbildungsmerkmal

Modell: Konkretes oder gedankliches Abbild eines vorhandenen Gebildes (~Realitätsausschnitt) oder Vorbild für ein zu schaffendes Gebilde in der Wahrnehmung der beteiligten Personen für einen bestimmten Verwendungszweck.

- Ein **Modell** stellt eine Abstraktion eines Realitätsausschnitts dar
  - Um Information verständlicher darzustellen
  - Um essentielle Aspekte aufzuzeigen
  - Zur Kommunikation (Projektmitarbeiter, Kunden)
  - Um komplexe Architekturen darstellen zu können

- Ein **Diagramm** ist die **grafische Repräsentation** eines Modells bzw. eines Modellausschnitts



Datenmodellarten

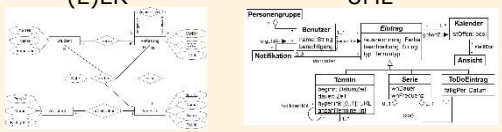
- Konzeptuelle (Analyse-/Entwurfs-)Modelle
  - **E**ntity-**R**elationship-Modell (ER)
    - generell einsetzbar
  - **E**rweiterte **E**ntity-**R**elationship-Modelle (EER)
    - erfassen zusätzliche Semantik
      - (min,max)-Notation, abgeleitete Attribute
      - Existenzabhängige Entity-Typen
      - Generalisierung und Spezialisierung
      - Aggregation
      - …
  - **U**nified **M**odeling **L**anguage (UML)
    - unterstützt objektorientierte Modellierung
    - deckt den gesamten Softwareentwurf ab
- Realisierungsmodelle
  - Relationenmodell
  - Objektorientierte Modelle
  - Semistrukturierte Modelle
  - …

© 2025

Datenbankentwurfsschritte

Systemvision

Konzeptuelles (Analyse-/Entwurfs-)Modell

Phase	Modell	Ergebnis
Anforderungs-analyse	Requirements Engineering	Pflichtenheft (Use-Cases, ...) Realitäts-ausschnitt **ANF 5.4:** Eine Vorlesung wird durch eine eindeutige Vorlesungsnummer identifiziert. ...
Konzeptueller Entwurf	Entity Relationship	Konzeptuelles DB-Schema Modell (E)ER UML 
Logischer Entwurf	Datenbankmodell (hierarchisch, objekt-orientiert, objekt-relational, semi-strukturiert, ...)	Logisches DB-Schema Vorlesung{VorlNr, Titel, ...} Student{MatrNr, Name, ...}
Physischer Entwurf	Data Definition Language	Physisches DB-Schema Oracle, DB2, SQLServer, MySql, ...

```
CREATE TABLE Vorlesung(
    VorlNr NUMBER(10) PRIMARY KEY,
    Titel VARCHAR(20),
    ...
);
```

Lauffähiges System

Überblick

- Konzeptuelle Datenmodellierung
  - Modellbegriff
  - Datenmodelle
  - Datenbankentwurfsschritte
- Objektorientierte Datenmodelle
  - Unified Modeling Language
  - Konzepte der Objektorientierung
- Klassendiagramm
  - Klassen- vs. Objektdiagramm
  - Klassen
  - Beziehungen

- Anhang: Definitionen

© 2025

Objektorientierte Datenmodelle
Ziele von UML

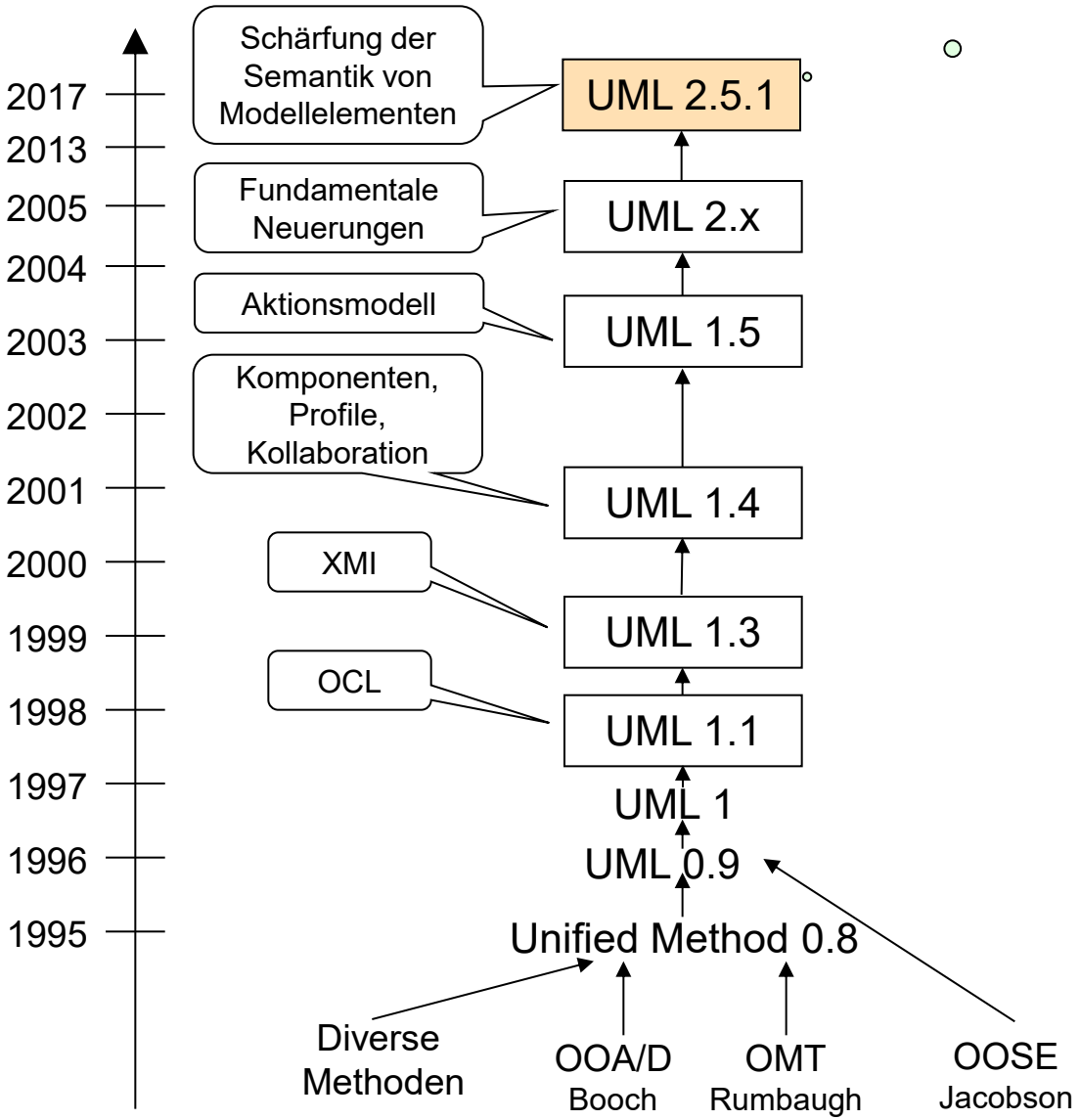
"Die Unified Modeling Language ist eine **visuelle Sprache** zur **Spezifikation**, **Konstruktion** und **Dokumentation** der **Artefakte von Software-Systemen**."

Object Management Group - OMG, www.uml.org

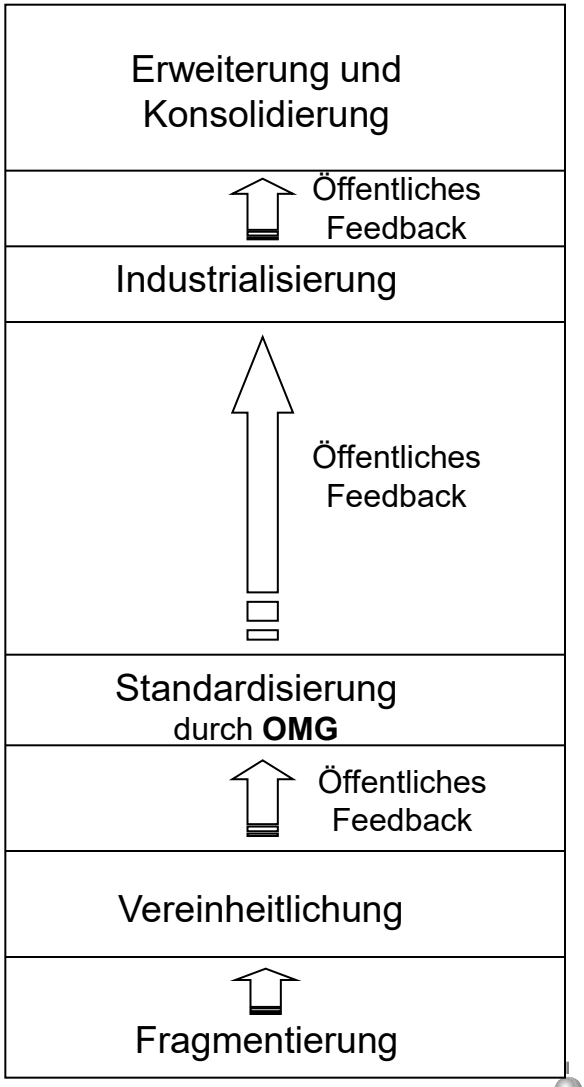
- Standardisierte, ausdrucksstarke, grafische **Modellierungssprache**, die auf **objektorientierten Konzepten** basiert
  - Für den Einsatz in verschiedenen Anwendungsbereichen
  - Zur Formulierung unterschiedlicher Modelltypen
  - Zum Austausch von Modellen

Objektorientierte Datenmodelle

Historische Entwicklung von UML

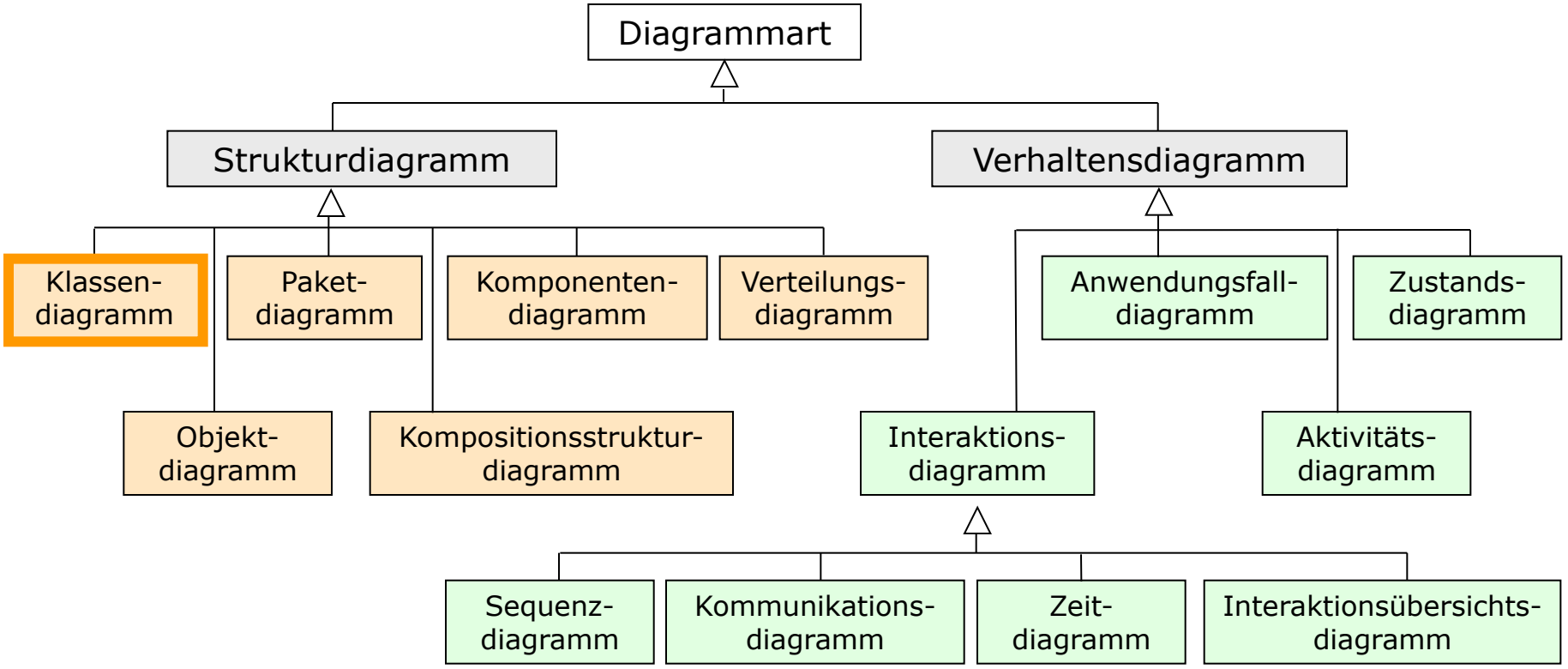


Model Driven Architecture



Objektorientierte Datenmodelle

UML-Diagrammarten



Klassendiagramme sind für den Datenbankentwurf relevant!

© 2025

UML für den Datenbankentwurf

- Fokus liegt auf **Klassendiagramme**
  - sind im Kern erweiterte ER-Diagramme (EER)
    - **Entity**-Typen entsprechen im allgemeinen der strukturellen Modellierung von **Klassen**
    - Menge gleichartiger **Entities** sind vergleichbar mit den **Objekten** einer Klasse
    - **Relationships** entsprechen **Assoziationen**
    - **Attribute** zur Beschreibung von Entitäten und Relationships auch in Klassen und Assoziationen vorhanden
    - **Funktionalitätsnotation** (Kardinalitäten) in ER entspricht **Multiplizitätsangabe** in UML
  - kennen jedoch kein **Schlüsselkonzept** (Wertsemantik)
  - stellen aber systemweit eindeutigen **Objektidentifikator** (OID) (Referenzsemantik) zur Verfügung
  - beschreiben Struktur **und Verhalten**
  - und stellen objektorientierte Konzepte wie **Vererbung**, **Polymorphie** sowie **Datenkapselung** bereit

© 2025

UML für den Datenbankentwurf

- Einsatz von Klassendiagrammen für die Datenbank-modellierung setzt voraus, dass diese
  - einerseits auf die in einem Datenbankmodell verwertbaren Konstrukte eingeschränkt werden
  - und andererseits um unverzichtbare Konzepte für ein Datenbankmodell erweitert werden.
- Darstellung eines konzeptuellen Datenbankmodells entspricht der allgemeinen Verwendung von Klassendiagrammen
- Darstellung eines physischen Datenbankmodells erfordert zusätzliche Modellierungselemente
- UML-Erweiterungsmechanismen (Profile, Stereotypen) erlauben entsprechende Anpassung des UML-Metamodells

© 2025

Überblick

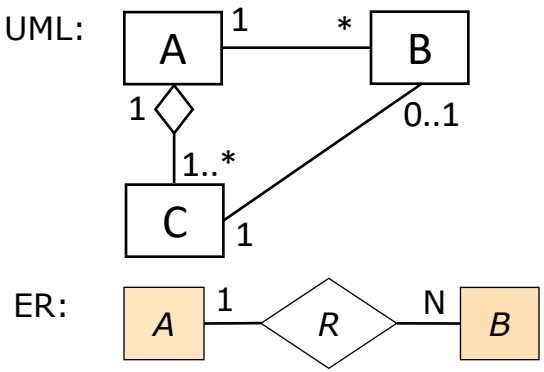
- Konzeptuelle Datenmodellierung
  - Modellbegriff
  - Datenmodelle
  - Datenbankentwurfsschritte
- Objektorientierte Datenmodelle
  - Unified Modeling Language
  - Konzepte der Objektorientierung
- Klassendiagramm
  - Klassen- vs. Objektdiagramm
  - Klassen
  - Beziehungen
    - Assoziation
    - Aggregation
    - Generalisierung
- Anhang: Definitionen

© 2025

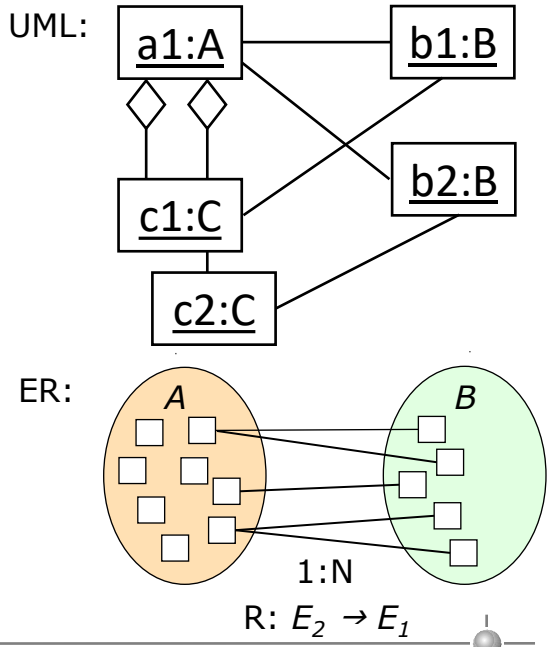
Klassendiagramm vs. Objektdiagramm

Einführung

- Klassendiagramm
  - Beschreibt den strukturellen Aspekt eines Systems auf Typebene in Form von Klassen, Interfaces und Assoziationen
  - Entspricht Datenbankschemanotation → beschreibt Typen von Instanzkollektionen



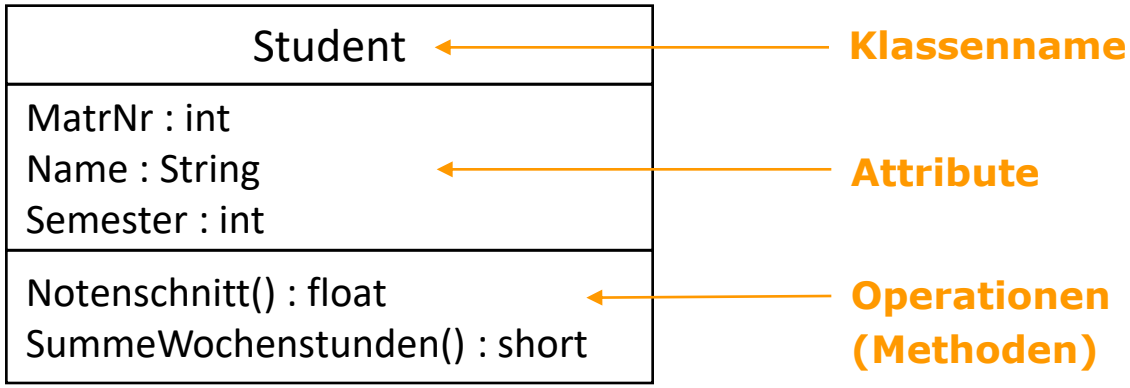
- Objektdiagramm
  - Beschreibt den strukturellen Aspekt eines Systems auf Instanzebene in Form von Objekten (= Instanz einer Klasse) und Links (= Instanz einer Assoziation)
  - Momentaufnahme des Systems – konkretes Szenario von Einzelobjekten
  - Ausprägung zu einem Klassendiagramm



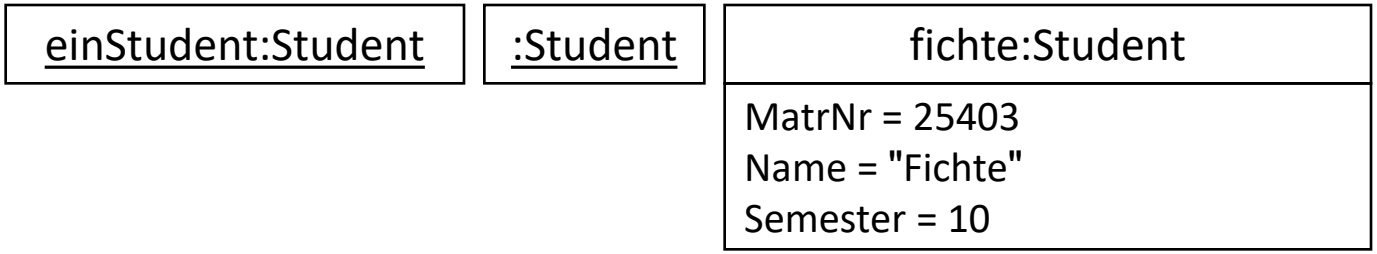
Klassendiagramm vs. Objektdiagramm

Einführung – Klasse und Objekt

- Klasse:

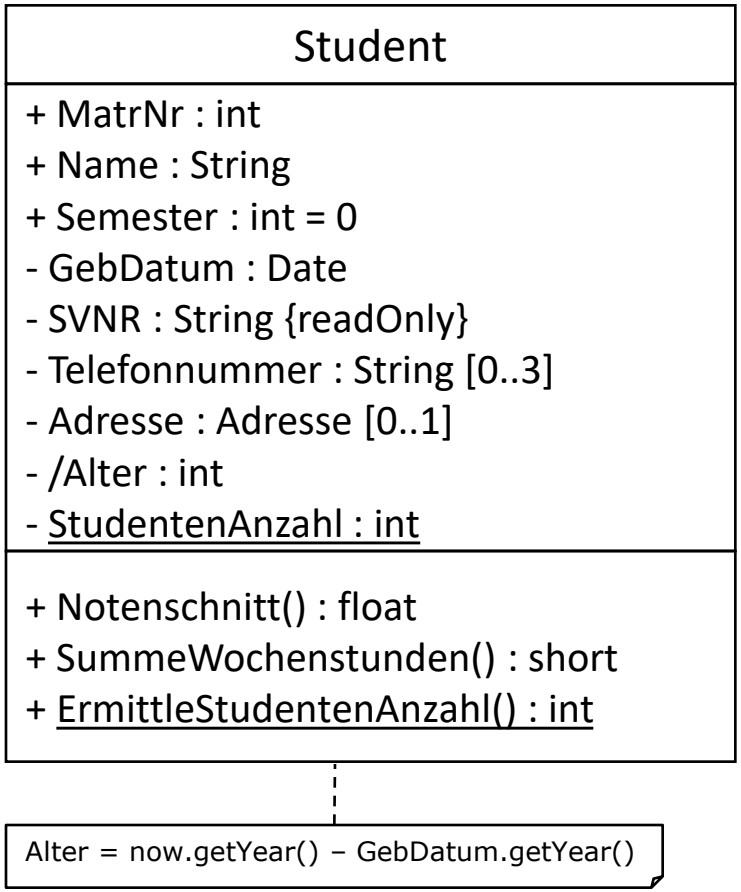


- Objekt:
  - Name des Objekts (und seiner Klasse) unterstrichen



Klassendiagramm

Klasse – Eigenschaften von Attributen und Operationen

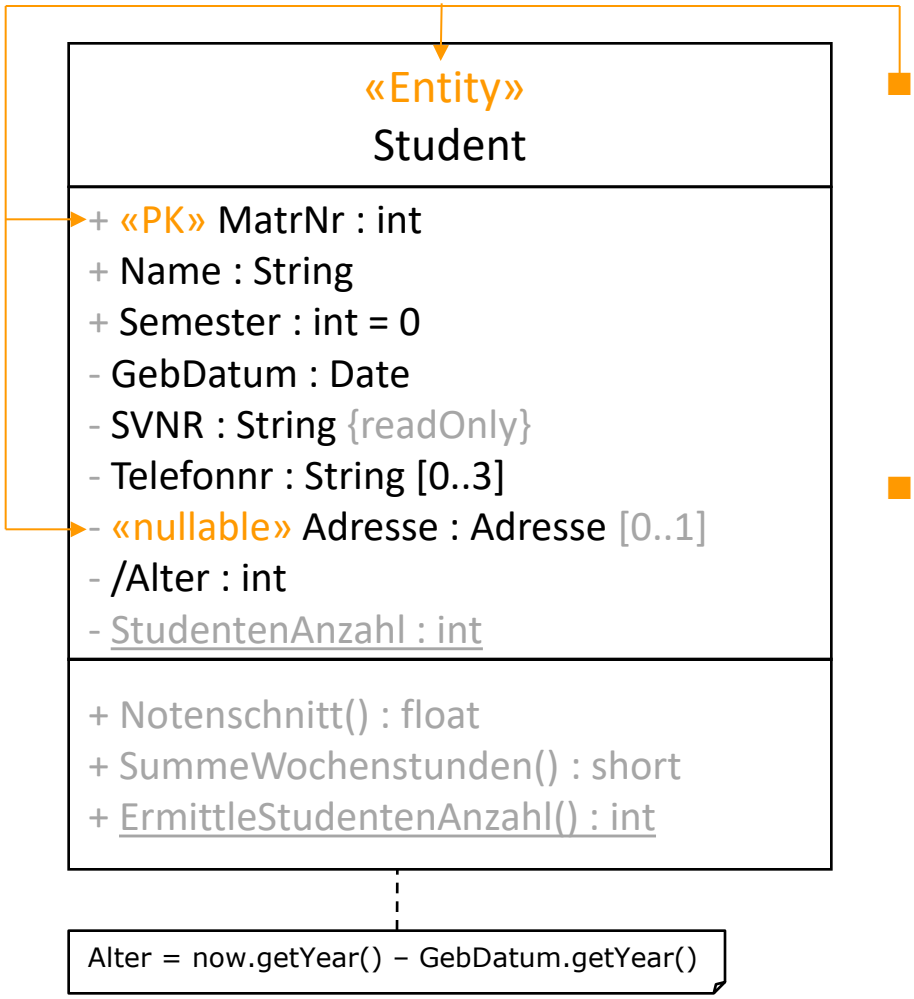


- Sichtbarkeiten von Attributen
  - \+ … public
  - \- … private
  - # … protected
  - ~ … package (vgl. Java)
- Eigenschaften von Attributen
  - /Attributname: abgeleitetes Attribut
  - : typ: Typangabe
  - [n..m]: Multiplizität von Attributen
  - = wert: Initialwert
  - Attributname: Klassenattribut
  - {readOnly}: Wert darf nach Initialisierung nicht verändert werden
- Berechnungsvorschrift
  - Notiz enthält Berechnung in natürlicher Sprache, in einer Programmiersprache oder in Pseudocode.

© 2025

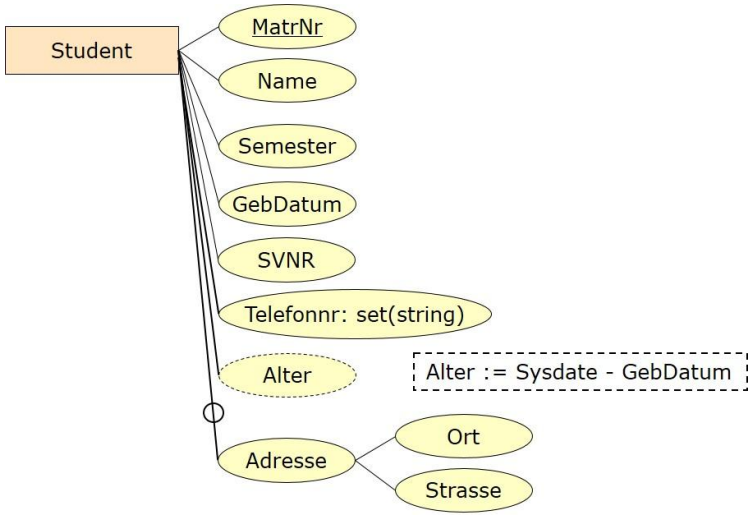
Klassendiagramm

Klasse – Anwendung in der Datenbankmodellierung



- **Stereotypen**
  - Erweiterungen von Modellelementen des UML-Metamodells um zusätzliche Metaattribute
  - Profile gruppieren Stereotypen für bestimmte Anwendungsbereiche (bspw. für die Datenbankmodellierung)

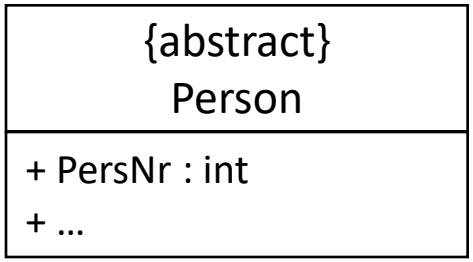
- Entitätsnotation



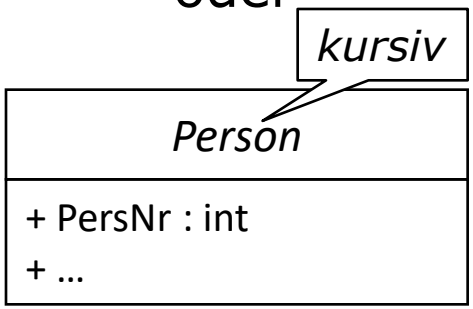
© 2025

Klassendiagramm

Klasse – Abstrakte Klasse



oder

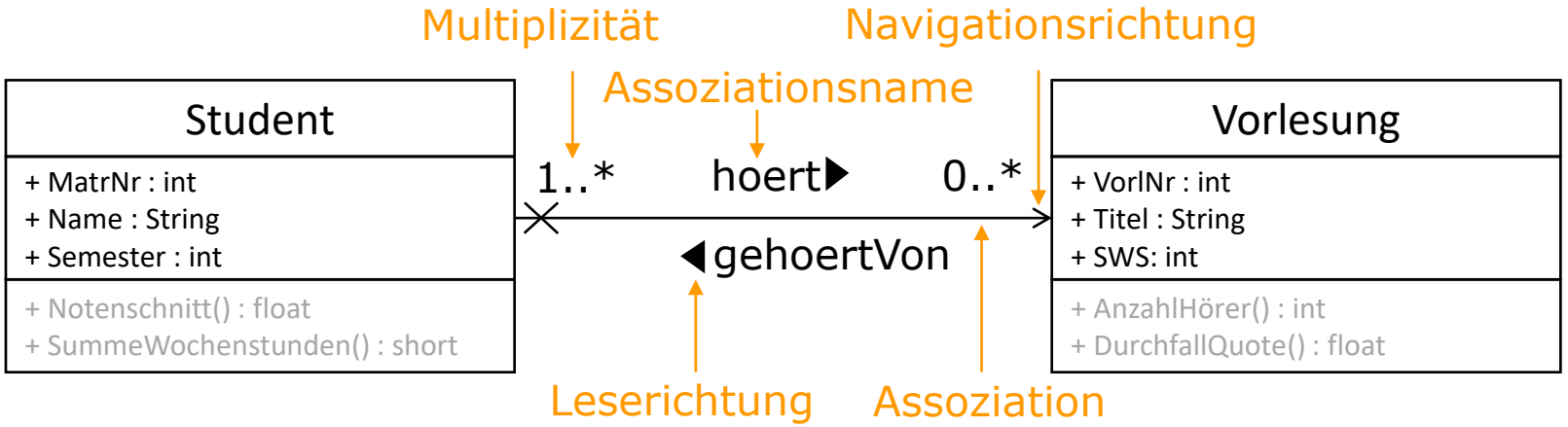


- Eine abstrakte Klasse
  - ist eine Klasse, die **nicht instanziiert** werden kann
  - ist daher **nur in Generalisierungs-hierarchien** sinnvoll
  - dient zum **Herausheben gemeinsamer Merkmale** einer Reihe von Unterklassen
- analog wird zwischen
  - konkreten (= implementierten) und
  - abstrakten (= nur spezifizierten)

  Operationen (Methoden) einer Klasse unterschieden

Klassendiagramm
Assoziation

- Assoziation zwischen Klassen modellieren (langfristige) Beziehungen zwischen den Objekten der Klassen



- Angaben (optional):
  - Assoziationsname
  - Pfeilspitze über Kante drückt Leserichtung aus
  - Pfeilspitze am Kantenende gibt Navigationsrichtung an
  - Multiplizität am Ende einer Assoziation: Mögliche Anzahlen jener Objekte, die mit genau einem Objekt der gegenüberliegenden Seite in Beziehung stehen können

© 2025

Klassendiagramm

Assoziation - Navigierbarkeit

- Eine gerichtete Kante gibt an, **in welche Richtung die Navigation** von einem Objekt zu seinem Partnerobjekt erfolgen kann
- Ein **nicht-navigierbares Assoziationsende** wird durch ein **»X«** am Assoziationsende angezeigt



  - Navigation von einem bestimmten Studenten zur entsprechenden Vorlesung
  - Umgekehrte Richtung – Welche Vorlesungen besucht ein bestimmter Student? – wird nicht unterstützt
- **Ungerichtete Kanten** bedeuten »**keine Angabe** über Navigationsmöglichkeiten«
  - In Praxis oft bidirektionale Navigierbarkeit angenommen
- Navigationsrichtung relevant für die Software Entwicklung aber **spielt bei der Datenbankmodellierung keine Rolle**

© 2025

Klassendiagramm

Assoziation - Multiplizität

- **Bereich**: »min .. max«
  - **ACHTUNG**: entspricht nicht der (min,max)-Notation im ER-Modell
- Beliebige **Anzahl**: »*« (= 0.. *)
- **Aufzählung** möglicher Kardinalitäten (Kommatrennung)

genau eines	1
beliebig	* oder 0..*
optional	0..1
fixe Anzahl (z.B.)	3
Bereich (z.B.)	3..* oder 3..6
Aufzählung	3,6,7,8,9 oder 3, 6..9

© 2025

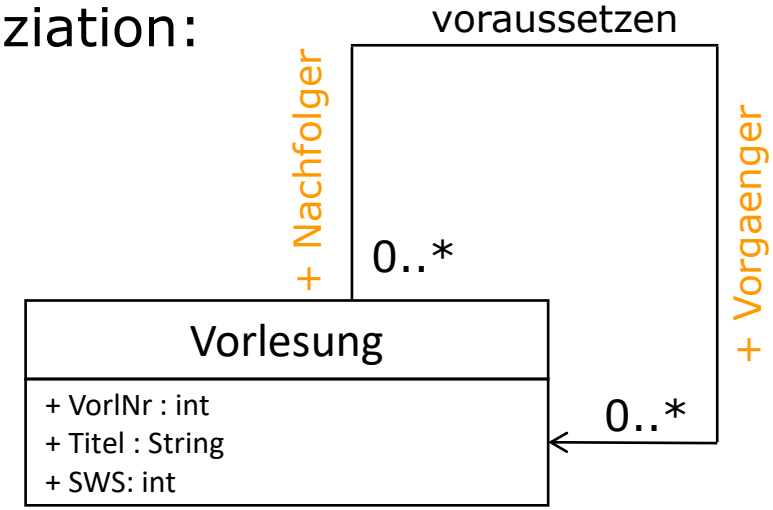
Klassendiagramm
Assoziation - Rollen

- Es können die **Rollen** festgelegt werden, die von den einzelnen Objekten in den Objektbeziehungen eingenommen werden

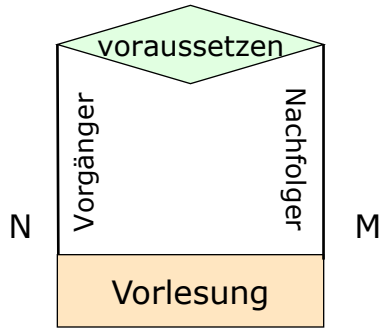
Binäre Assoziation:



Unäre Assoziation:

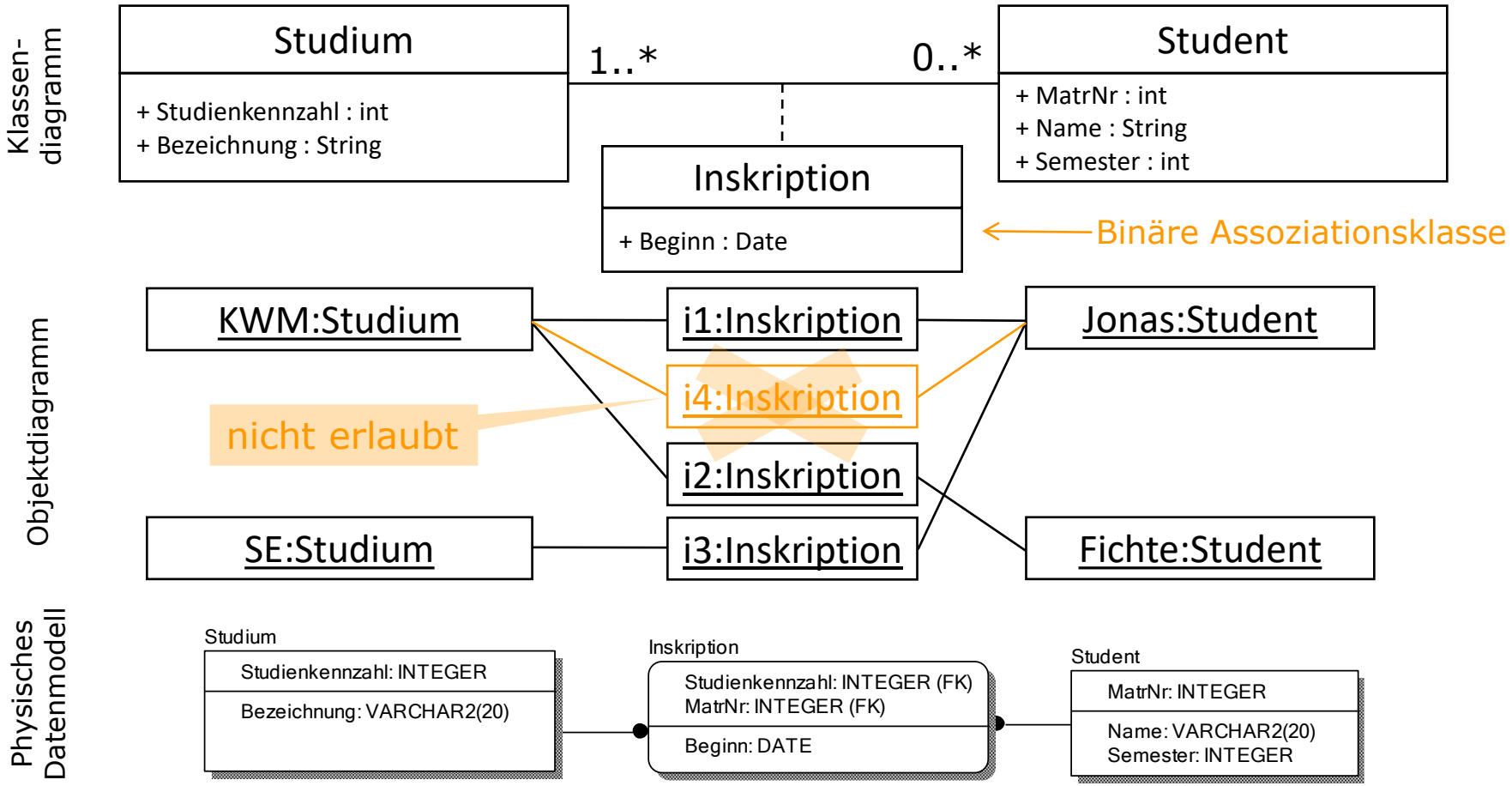


ER-Notation:



Klassendiagramm
Assoziationsklasse

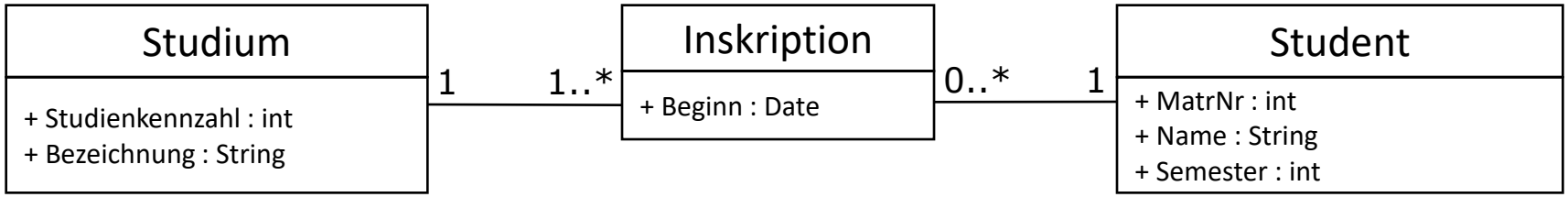
- Kann **Attribute der Assoziation** enthalten
  - Bei m:n-Assoziationen mit Attributen notwendig



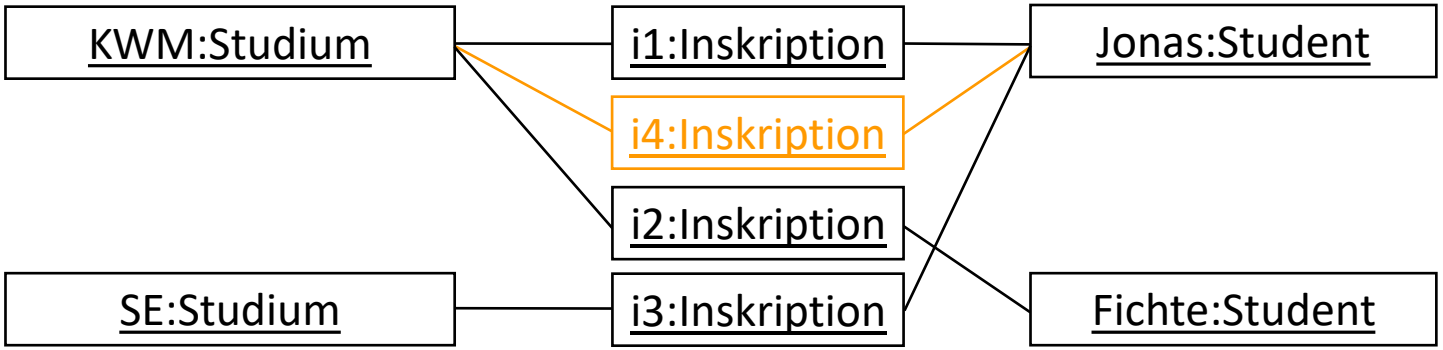
Klassendiagramm

„Gewöhnliche" Klasse

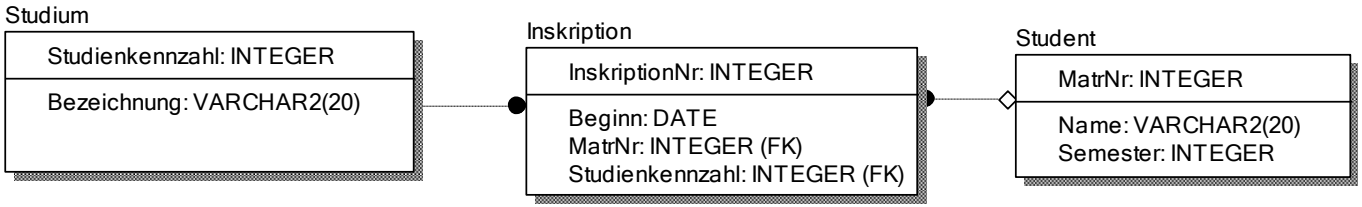
Klassen-diagramm



Objektdiagramm



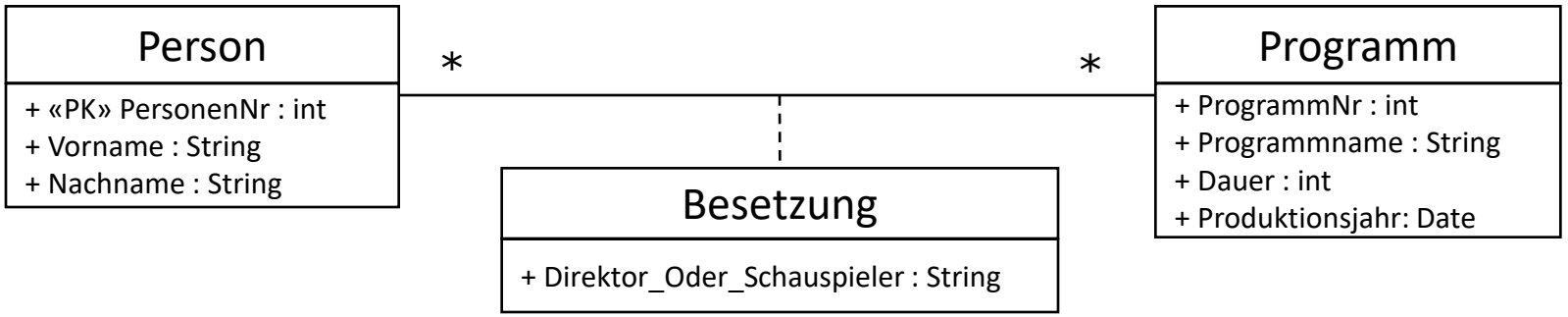
Physisches Datenmodell

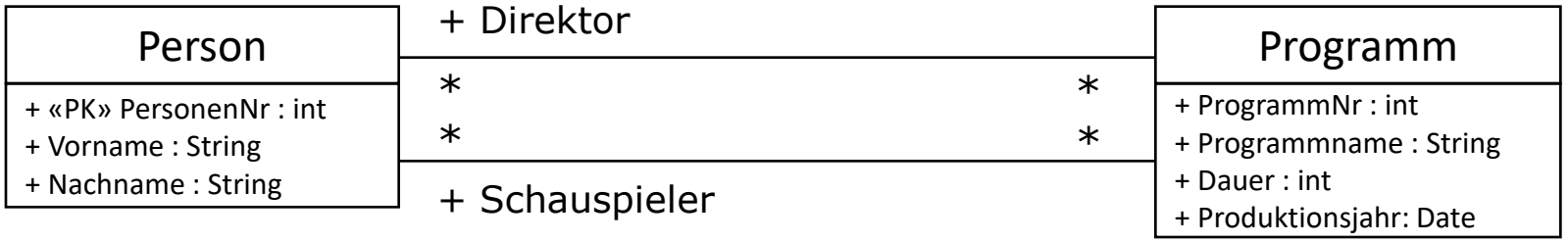


Aufgabe



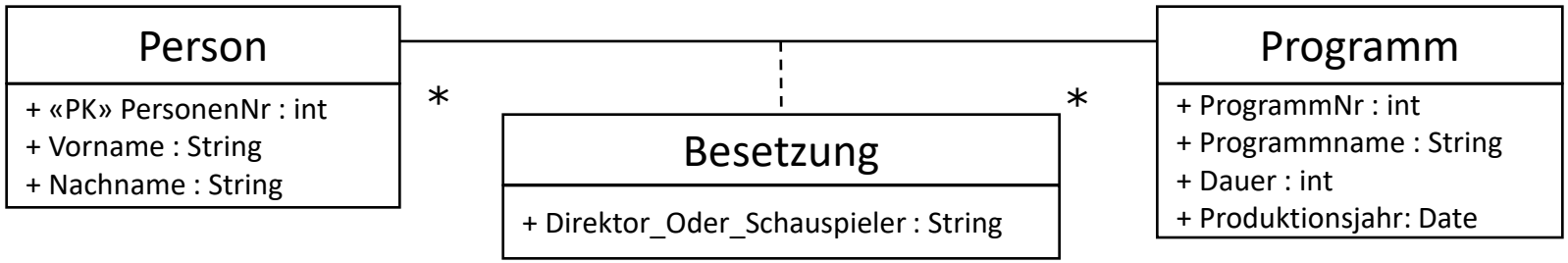
- Folgende zwei UML-Klassendiagramme modellieren auf unterschiedliche Weise den Sachverhalt TV Programm, DirektorIn und SchauspielerIn.

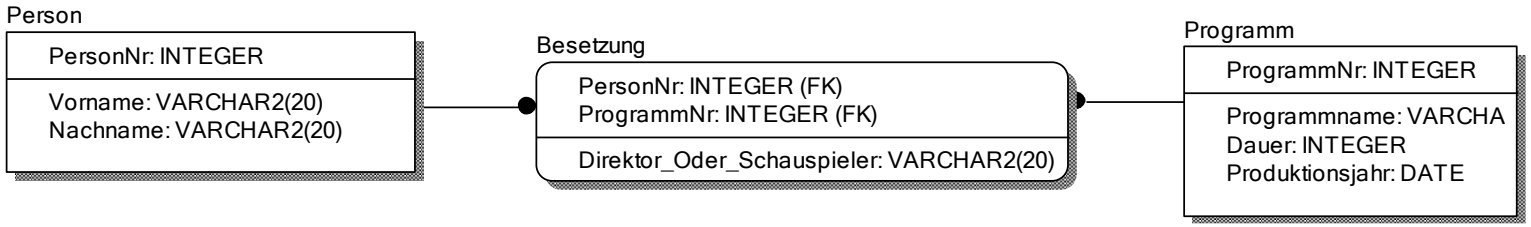


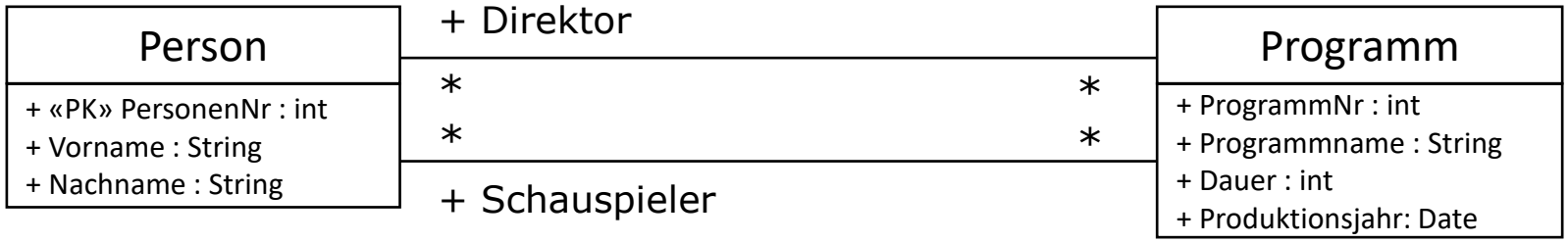


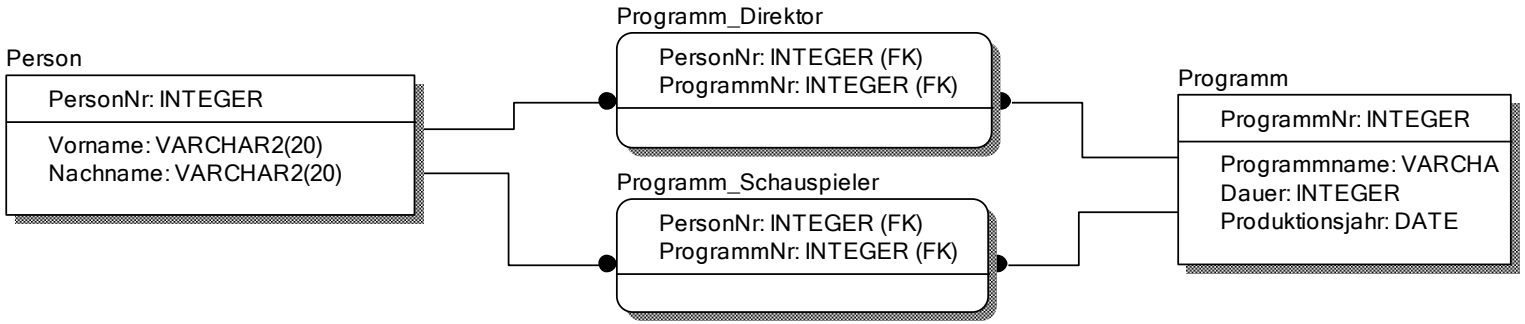
- Worin liegt der Unterschied in der physischen Datenspeicherung beider Datenmodelle?

Aufgabe





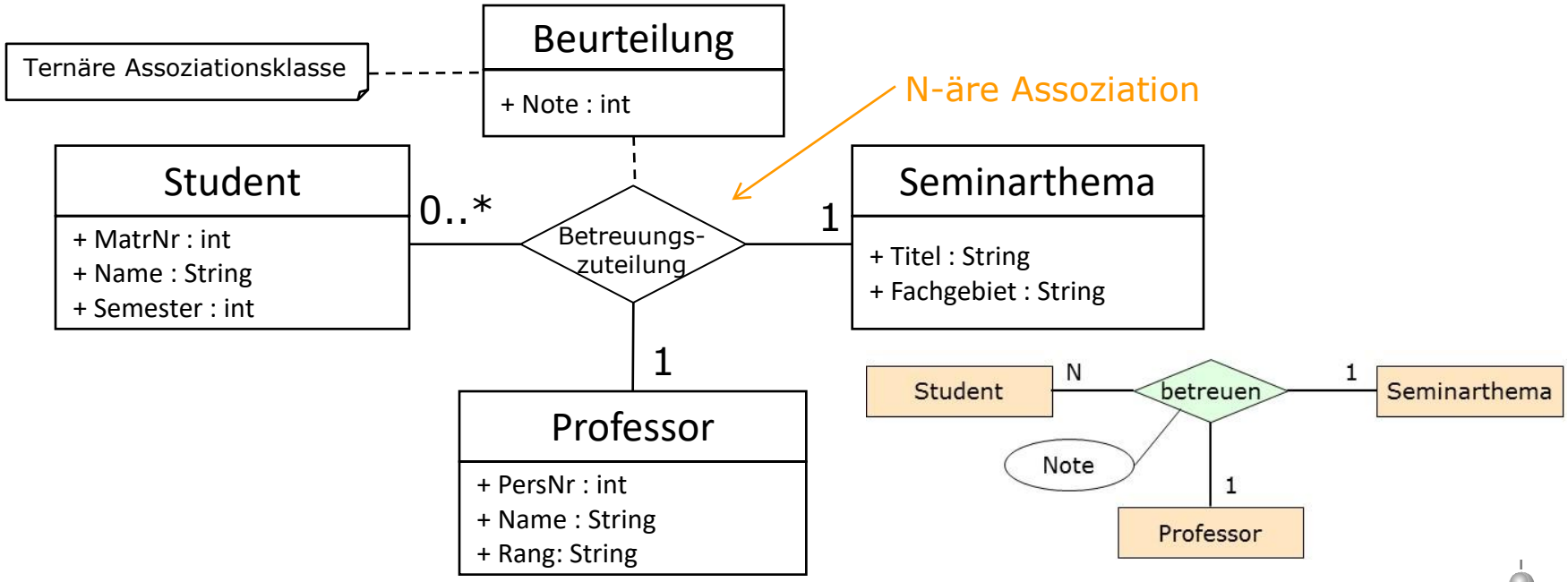




© 2025

Klassendiagramm
N-äre Assoziation

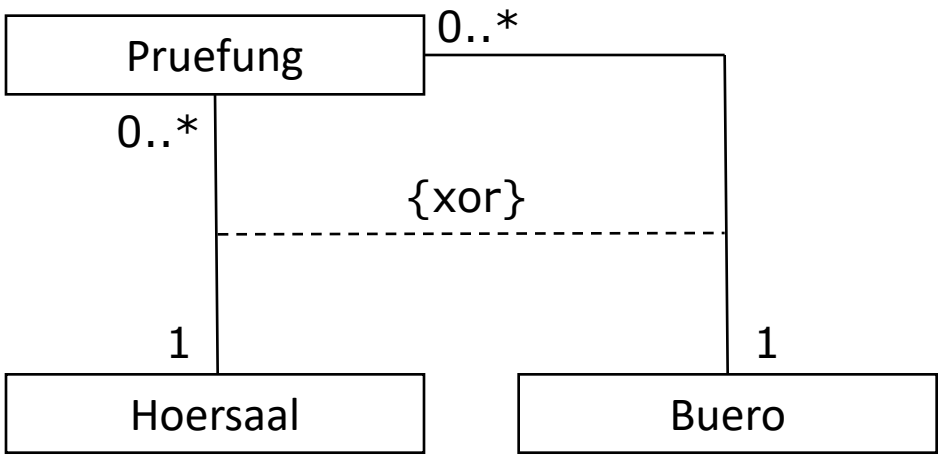
- Beziehung zwischen **mehr als zwei Klassen**
  - Navigationsrichtung kann nicht angegeben werden
  - Multiplizitäten geben an, wie viele Objekte einer Rolle/Klasse einem festen (n-1)-Tupel von Objekten der anderen Rollen/Klassen zugeordnet sein können
- Beispiel für eine ternäre (3-wertige) Assoziation:



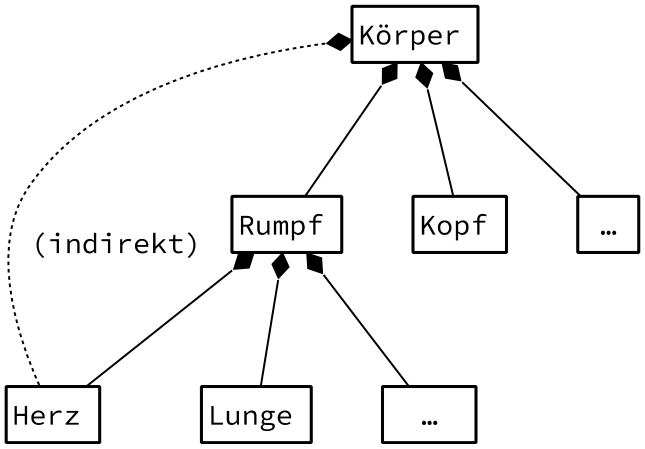
Klassendiagramm

Assoziation – Exklusivität

- Exklusives Oder {xor}
  - Für ein bestimmtes Objekt kann zu einem bestimmten Zeitpunkt nur eine von verschiedenen möglichen Assoziationen instanziiert werden



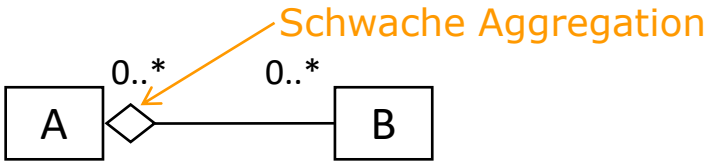
Klassendiagramm
Aggregation



- Aggregation ist eine **spezielle (stärkere) Form der Assoziation** mit folgenden Eigenschaften:
  - Transitivität:
    A ist Teil von B ∧ B ist Teil von C ⇒ A ist Teil von C
    Herz ist Teil von Rumpf ∧ Rumpf ist Teil von Körper
    ⇒ Herz ist (indirekt) Teil von Körper
  - Anti-Symmetrie:
    A ist Teil von B ⇒ B ist **nicht** Teil von A
    Herz ist Teil von Rumpf ⇒ Rumpf ist **nicht** Teil von Herz
- Aggregation wird auch als **"Teile-Ganzes-"** oder **"Besteht-aus-Beziehung""** bezeichnet.
- UML unterscheidet **zwei Aggregationsarten**:
  - Schwache Aggregation (shared aggregation)
  - **Starke Aggregation** (= Komposition) (composite aggregation)

© 2025

Klassendiagramm
Schwache Aggregation

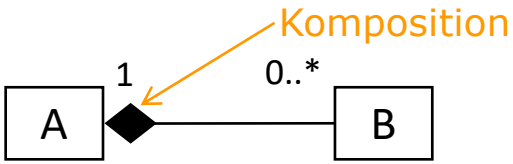


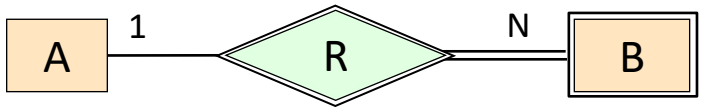
- Schwache Zugehörigkeit der Teile, d.h. Teile sind **unabhängig** von ihrem Ganzen
- Die Multiplizität des aggregierenden Endes der Beziehung (Raute) kann > 1 sein
- Es gilt nur eingeschränkte **Propagierungssemantik**
- Die zusammengesetzten Objekte bilden einen **gerichteten, azyklischen** Graphen



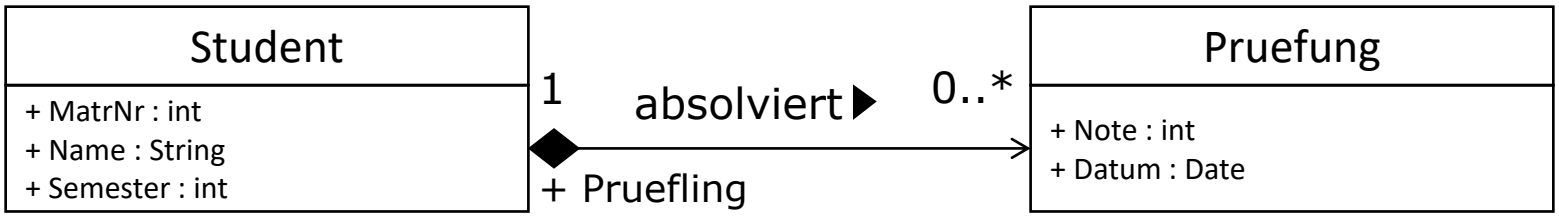
Klassendiagramm

Starke Aggregation - Komposition



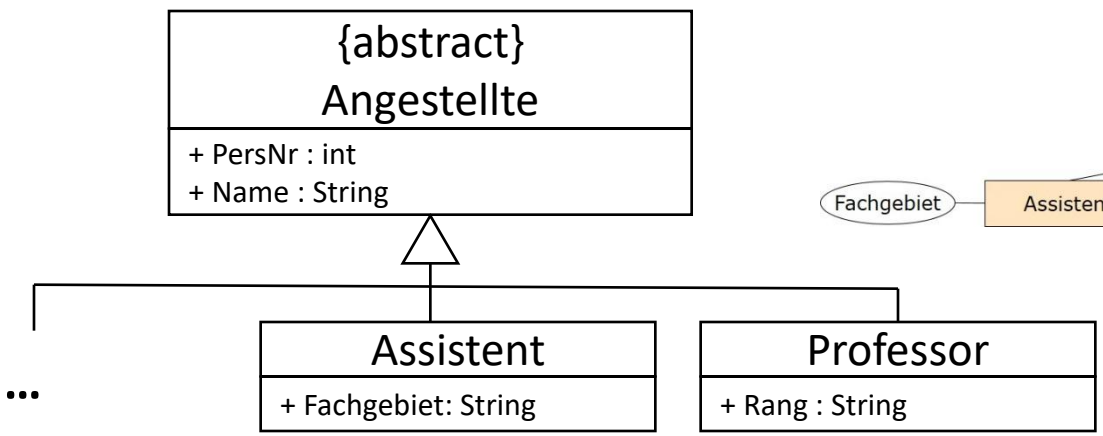


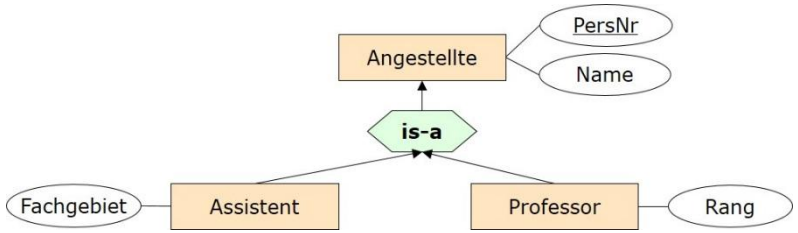
- Ein bestimmter Teil darf zu einem bestimmten Zeitpunkt in **maximal einem zusammengesetzten Objekt enthalten** sein.
- Die **Multiplizität** des aggregierenden Endes der Beziehung kann (maximal) 1 sein.
- **Abhängigkeit** der Teile vom zusammengesetzten Objekt
- **Propagierungssemantik**: Kopier- u. Löschoperationen
- Die zusammengesetzten Objekte bilden einen **Baum**



Klassendiagramm
Generalisierung/Spezialisierung

- Generalisierung
  - Eigenschaften verwandter Klassen werden herausgehoben und einer **Oberklasse** zugeordnet.
  - Oberklasse hat die gemeinsamen Attribute der abgeleiteten Klassen.
- Spezialisierung
  - **Unterklassen** werden von Oberklassen abgeleitet und
  - haben neben den vererbten Attributen auch eigene Attribute.



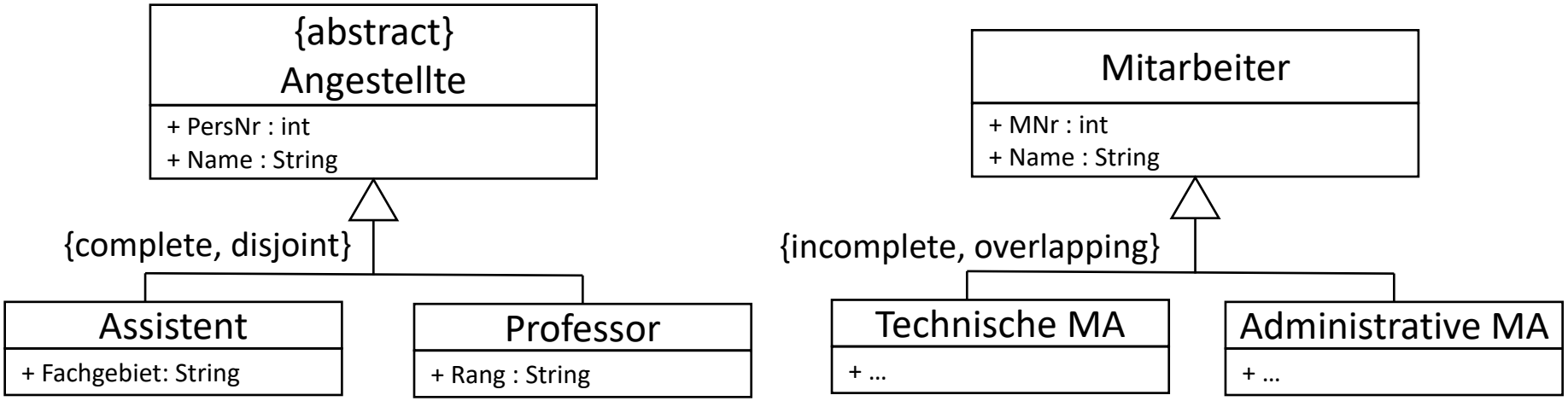


- Wird in UML als **Vererbung** zusammengefasst

Klassendiagramm

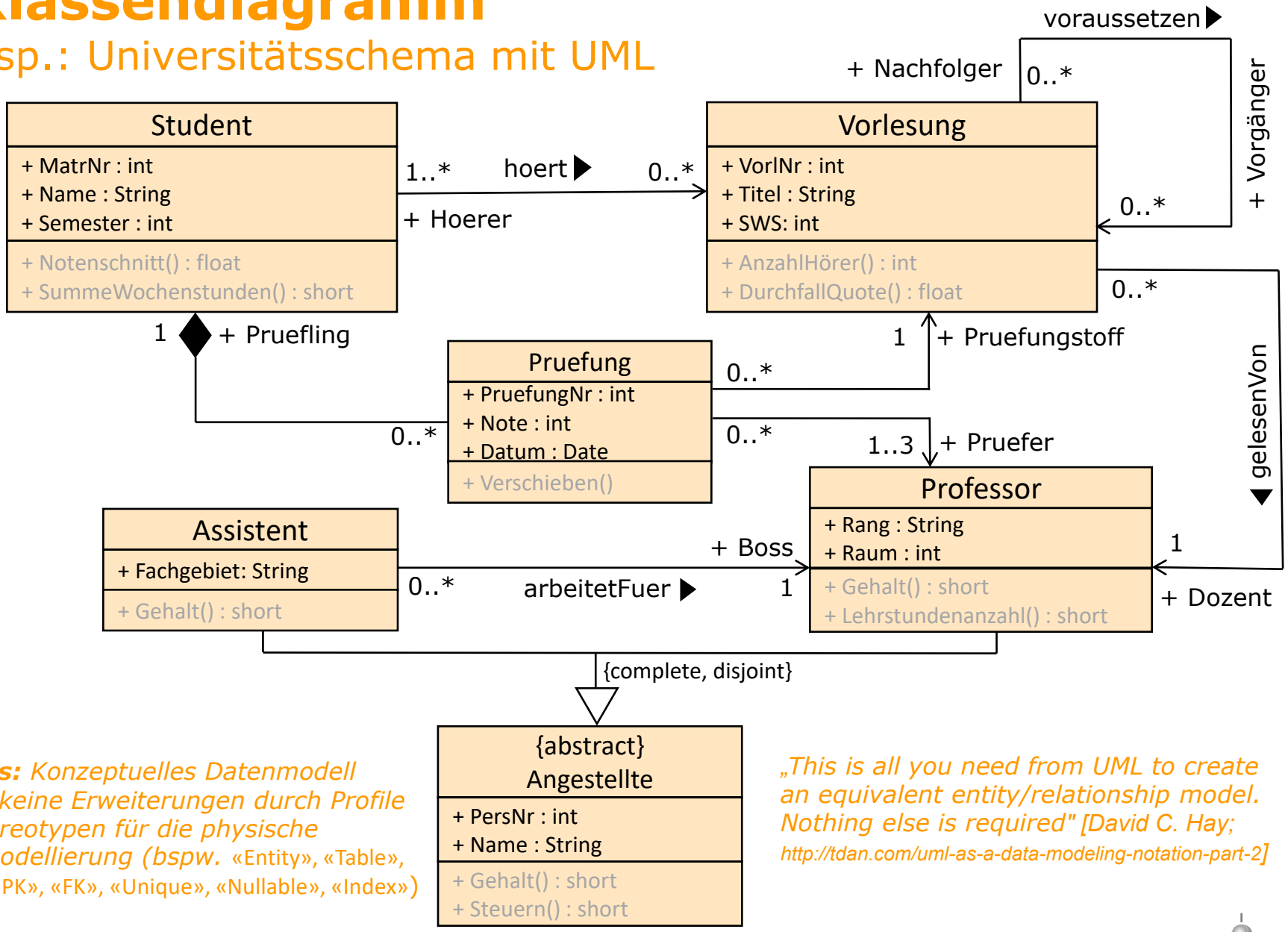
Generalisierung - Eigenschaften

- Unterscheidung kann vorgenommen werden in
  - unvollständig / vollständig
    vollständig (default): jede Instanz der Oberklasse muss auch Instanz mindestens einer Unterklasse sein
  - überlappend / disjunkt
    disjunkt (default): Objekt kann Instanz von maximal einer Unterklasse sein



Klassendiagramm

Bsp.: Universitätsschema mit UML

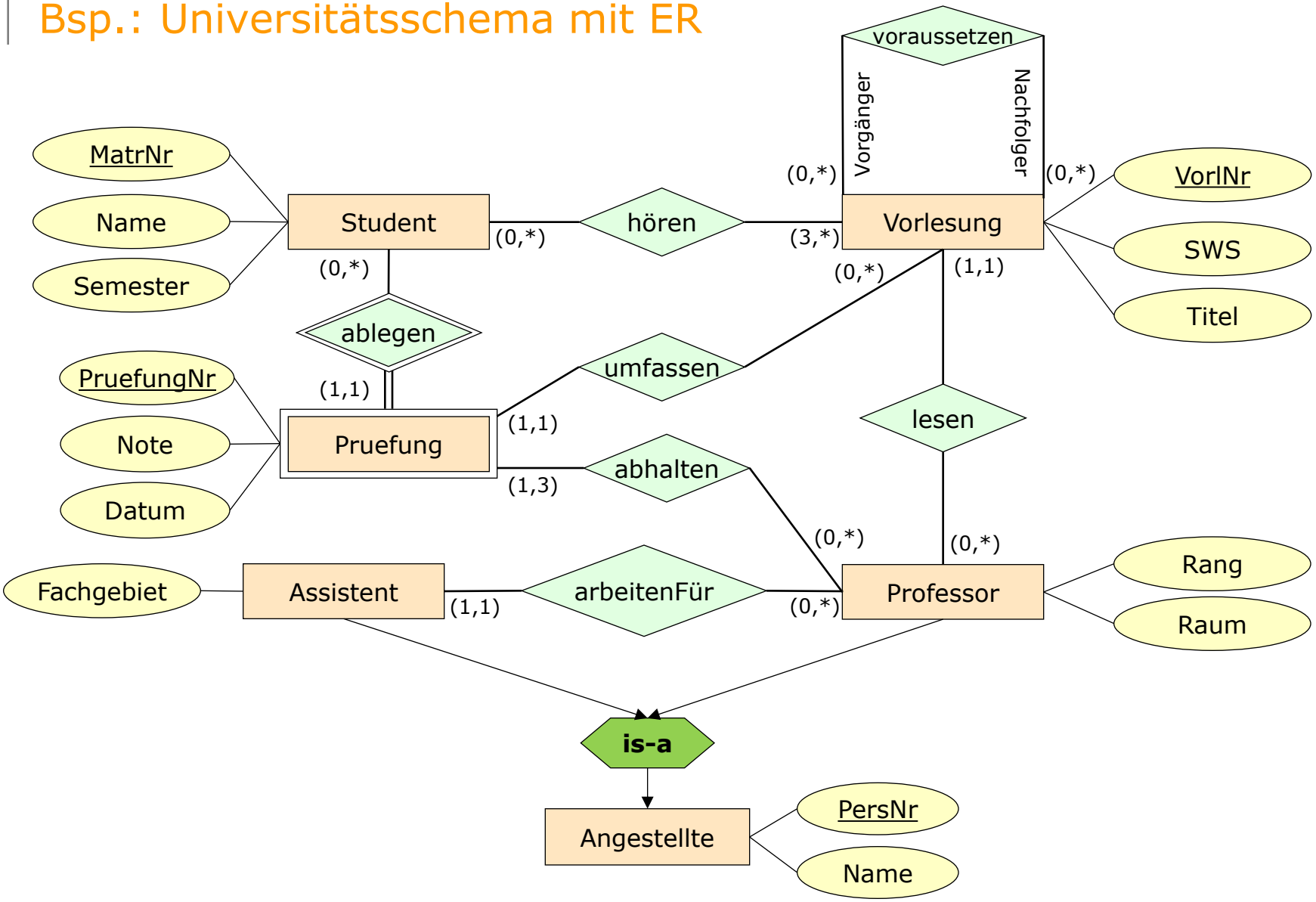


Hinweis: *Konzeptuelles Datenmodell enthält keine Erweiterungen durch Profile und Stereotypen für die physische Datenmodellierung (bspw.* «Entity», «Table», «View», «PK», «FK», «Unique», «Nullable», «Index»)

„This is all you need from UML to create an equivalent entity/relationship model. Nothing else is required" [David C. Hay; http://tdan.com/uml-as-a-data-modeling-notation-part-2]

Klassendiagramm

Bsp.: Universitätsschema mit ER



Datenmodellierungskonzepte

(E)ER versus UML

	(E)ER	UML Klassendiagramm
ER	Entity	Klasse
	Eigenschaft mit Wertebereich (Domäne)	Attribut mit Datentyp
	Relationship	Assoziation
	Primärschlüssel	Keine direkte Entsprechung, da Objekte automatisch einen OID aufweisen; mittels UML-Profilen aber abbildbar (z.B. Stereotyp «PK»)
EER	Kardinalitäten – (min,max)-Notation	Multiplizitäten
	Abgeleitete Attribute	Abgeleitete Attribute
	Existenzabhängige Entity-Typen	Komposition
	Aggregation	Aggregation
	Generalisierung und Spezialisierung	Vererbung

© 2025

Zusammenfassung

"Data model quality is not a function of the notation selected! Data model quality is a function of
- *Clarity of thought*
- *Clarity of presentation" [David C. Hay]*

- ER-Diagramme sind eine **weit verbreitete** Art der konzeptuellen Datenbankmodellierung
  - Konzepte sind **nicht allzu schwer zu begreifen**, d.h. der Sachverhalt kann auch mit "Informatiklaien" (die den Fachbereich kennen) diskutiert werden
  - Wegen Einschränkungen vermehrt Verdrängung der ER-Diagramme durch **objektorientierte Modelle (UML)**
- UML-Klassendiagramme unterstützen **objektorientierte Konzepte**
  - standardisiert und ausdruckstärker
  - Spezifikation von Struktur und Verhalten (Methoden)
  - genauere Kardinalitätsrestriktionen (Multiplizitäten)
  - Unterstützung der Abstraktionskonzepte der Generalisierung/Spezialisierung, Aggregation/Komposition
  - UML deckt den gesamten Softwareentwurf ab

© 2025

Anhang: (E)ER versus UML

	EER	UML Klassendiagramm
Zweck	Konzeptionelle Modellierungssprache für **DB-Schemata** (DSL – Domain-specific language)	Konzeptionelle Modellierungssprache für **objektorientierte Softwaresysteme** (GPL – General-purpose language)
Ausdrucksstärke	▪ Reine **Struktur-modellierungssprache** ▪ In Urversion nicht so ausdrucksstark (z.B. keine Vererbung)	▪ Nicht auf DB-Schemata beschränkt (viele Diagrammarten → kann **Struktur** und **Verhalten** modellieren) ▪ Kann damit alles abbilden, was auch mit EER abbildbar ist und noch viel darüber hinaus (z.B. Verhalten!)
Entwicklungshistorie	„Alt" → Urversion aus 1976; später Erweiterungen (EER)	UML Version 0.9 aus 1996; zahlreiche Weiterentwicklungen bis aktuell UML Version 2.5.1 aus 2017
Entwicklungsstatus	Nicht standardisiert	**Standardisiert**; de facto Modellierungssprache im Softwareentwicklungsbereich
Verbreitung	Auf DB-Entwurf beschränkt, daher Werkzeugunter-stützung nicht so groß wie bei UML	Große Werkzeugunterstützung

© 2025

Anhang: Definitionen

Klassendiagramm

UML-Pendant zum erweiterten ER-Schema

Klasse

Beschreibung der Struktur (Attribute) und des Verhaltens (Operationen/Methoden) einer Menge von Objekten.

Abstrakte Klasse

Klasse, die nicht instanziiert werden kann.

Assoziation

Beziehung zwischen Klassen: Angabe über Navigationsrichtung kann erfolgen.

N-äre Assoziation

Beziehung zwischen n Klassen.

Assoziationsklasse

Nähere Beschreibung einer Assoziation mit Attributen.

© 2025

Anhang: Definitionen

xor-Beziehung

Ein Objekt von C steht entweder mit einem Objekt von A oder mit einem Objekt von B in Beziehung, nicht aber mit beiden.

Starke Aggregation = Komposition

Existenzabhängige Teile-Ganzes-Beziehung (A ist Teil von B; wenn B gelöscht wird, werden zugehörige Instanzen von A ebenfalls gelöscht)

Schwache Aggregation

Teile-Ganzes-Beziehung (A ist Teil von B; wenn B gelöscht wird, werden zugehörige Instanzen von A nicht gelöscht.

Generalisierung

Vererbungsbeziehung zwischen Klassen (A erbt von B)

Objekt

Instanz einer Klasse

Link

Beziehung zwischen Objekten

© 2025